

Season 1: How LLMs work

One token at a time.

What a large language model (LLM) actually does

Every LLM behaviour starts here

- **Cost and latency:** output is produced one token at a time
- **Memory:** every call starts from nothing
- **Accuracy:** models are trained to be plausible, not correct

An LLM is a stateless function in a loop

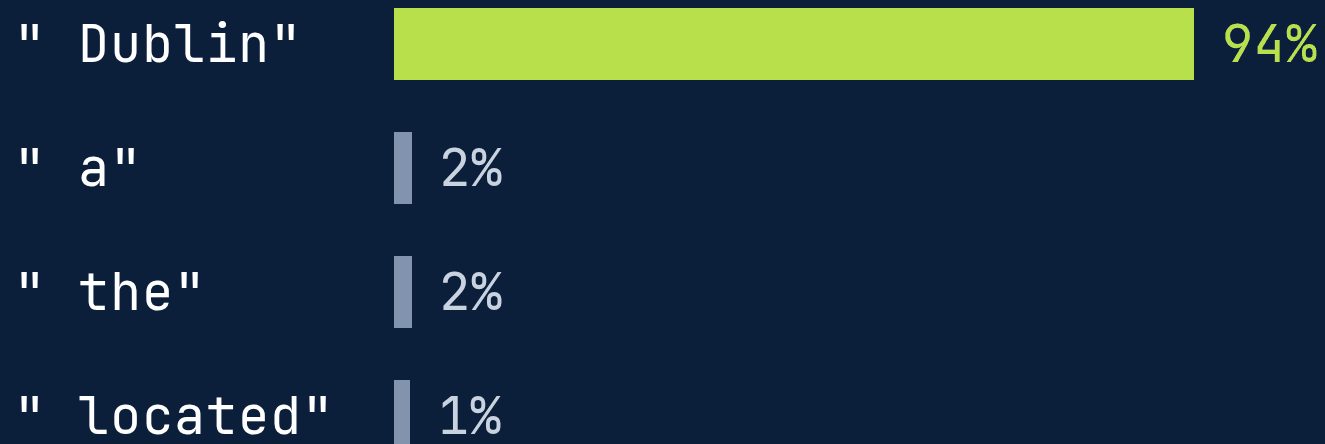
It takes the whole token sequence and returns a probability for every possible next token. Append one, call again.

Where it breaks

A pure function is deterministic. Sampling makes LLM output vary, even for identical input.

It predicts the next token

Input: "The capital of Ireland is"

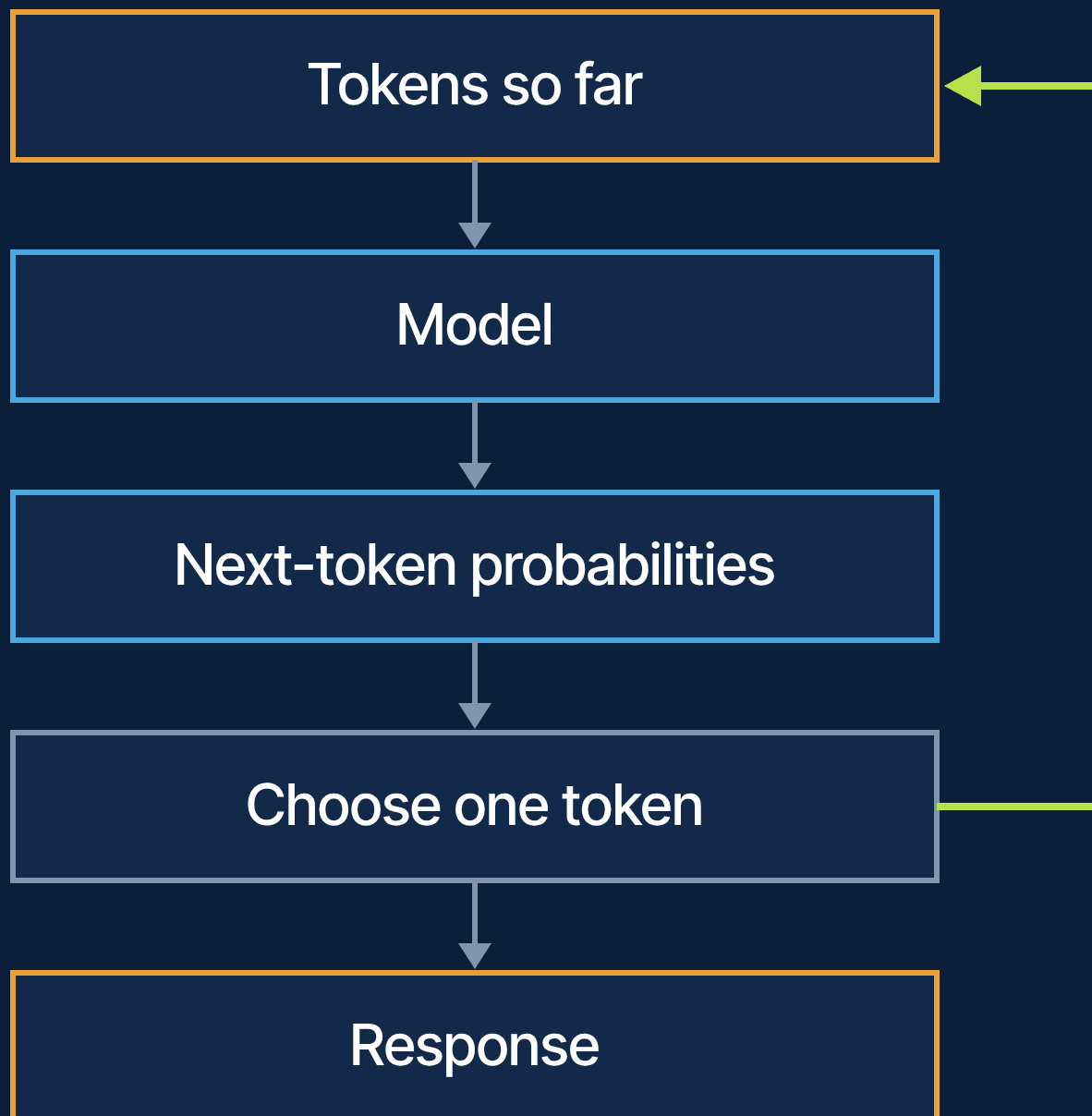


 The most likely next token

The model scores every token in its vocabulary, then one is chosen. That is the entire output of one step.

Illustrative probabilities. S1 E8 covers how the token is chosen.

The generation loop



□ Append the token and run the model again

Generation stops at an end token or the output limit.

How a model is made

Pretraining

Predicts the next token across vast amounts of text, learning language patterns and broad knowledge

Instruction tuning

Trains on examples of instructions and good answers, turning a text predictor into an assistant

Preference tuning

Reinforcement learning from human feedback (RLHF) and similar methods favour helpful, safe answers

Methods and stages vary by provider.

Trained to be plausible, not correct

- Pretraining rewards the likely next token, not the true one
- A fluent, confident answer can still be wrong
- Reliable answers come from grounding and evaluation, not the model alone

S1 E9 covers hallucination. Season 2 covers retrieval.

Calling a model

OpenAI's Python library, as of September 2026.

```
from openai import OpenAI

# Reads OPENAI_API_KEY from the environment
client = OpenAI()

first = client.responses.create(
    model="gpt-6-astra",
    input="My name is Brendan.",
)
print(first.output_text)
print("Input tokens:", first.usage.input_tokens)
print("Output tokens:", first.usage.output_tokens)
```

■ Every response reports the tokens it used

Output tokens include hidden reasoning tokens. S1 E2 explains why.

It does not remember you

```
second = client.responses.create(  
    model="gpt-6-astra",  
    input="What is my name?",  
)  
print(second.output_text)
```

What comes back

The model has no way to know your name. Nothing from the first call was sent.

To continue a conversation, send the history or link calls with a provider feature such as `previous_response_id`. S1 E2 shows the cost. Full script in the [applied-ai-lab repository](#).

What breaks in production

- **Assuming memory:** history must be sent, or linked, on every call
- **Assuming determinism:** identical prompts can return different answers
- **Assuming truth:** fluent answers can be confidently wrong
- **Unbounded output:** every call needs an output limit

An LLM predicts. It does not look things up.

Reliability comes from the system around the model: the data it is given, evaluation, and guardrails.

Ask your team

Where does this feature depend on the model being right, and how do we check it?

Decision rule

Treat every model call as stateless and probabilistic: send the context it needs, and verify what it returns.

Next: S1 E2 Tokens

Why cost, limits, and language all come down to tokens.

Where this fits

The series map

S1 LLMs	S2 Retrieval	S3 Strategy
S4 Adapting	S5 Agents	S6 Economics
S7 Data	S8 Evaluation	S9 AI teams
S10 Reliability	S11 Security	S12 Delivery
S13 Performance	S14 Resilience	S15 Governance

☐ Leadership season

 Follow	linkedin.com/in/brendan-kelly-irl
 Episode guide	brendankellyai.github.io
 Code	github.com/BrendanKellyAI/applied-ai-lab